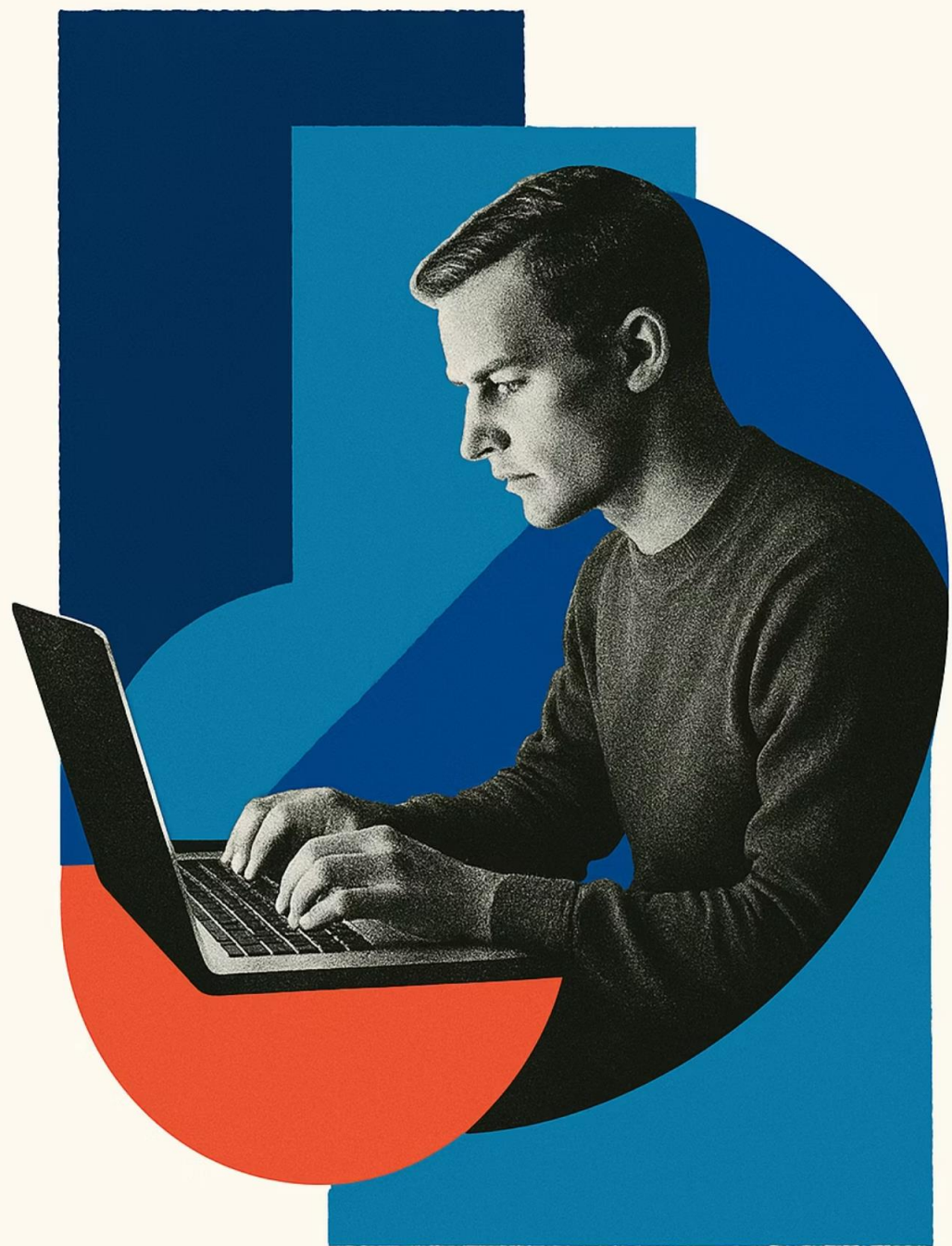


Angular Moderno

08 - Introdução a Signals



08 — Introdução a Signals

Angular Moderno

O que vamos aprender hoje?

01

Entender o problema

Por que variáveis comuns não são suficientes em aplicações reativas modernas.

02

Conhecer Signals

O que são, como funcionam e por que o Angular adotou esse modelo.

03

Aplicar na prática

Transformar o nosso projeto de e-commerce para usar `signal()` passo a passo.

04

Evitar erros comuns

Reconhecer os erros mais frequentes e saber como corrigi-los.

Objetivo da Aula

Meta principal

Aprender o conceito de **Signals** no Angular moderno e aplicar diretamente no projeto existente, substituindo um estado comum por um **estado reativo** utilizando `signal()`.

Ao final desta aula, o nosso projeto exibirá novos produtos dinamicamente — sem RxJS, sem gambiarras.

O que você vai conseguir fazer

- Converter um array comum em um `signal()`
- Usar `update()` para modificar o estado com segurança
- Atualizar o template para ler o signal corretamente
- Ver o Angular reagir automaticamente às mudanças

Onde Estamos no Projeto?

Antes de avançar, é importante saber exatamente o que já construímos juntos até aqui.

Componentes

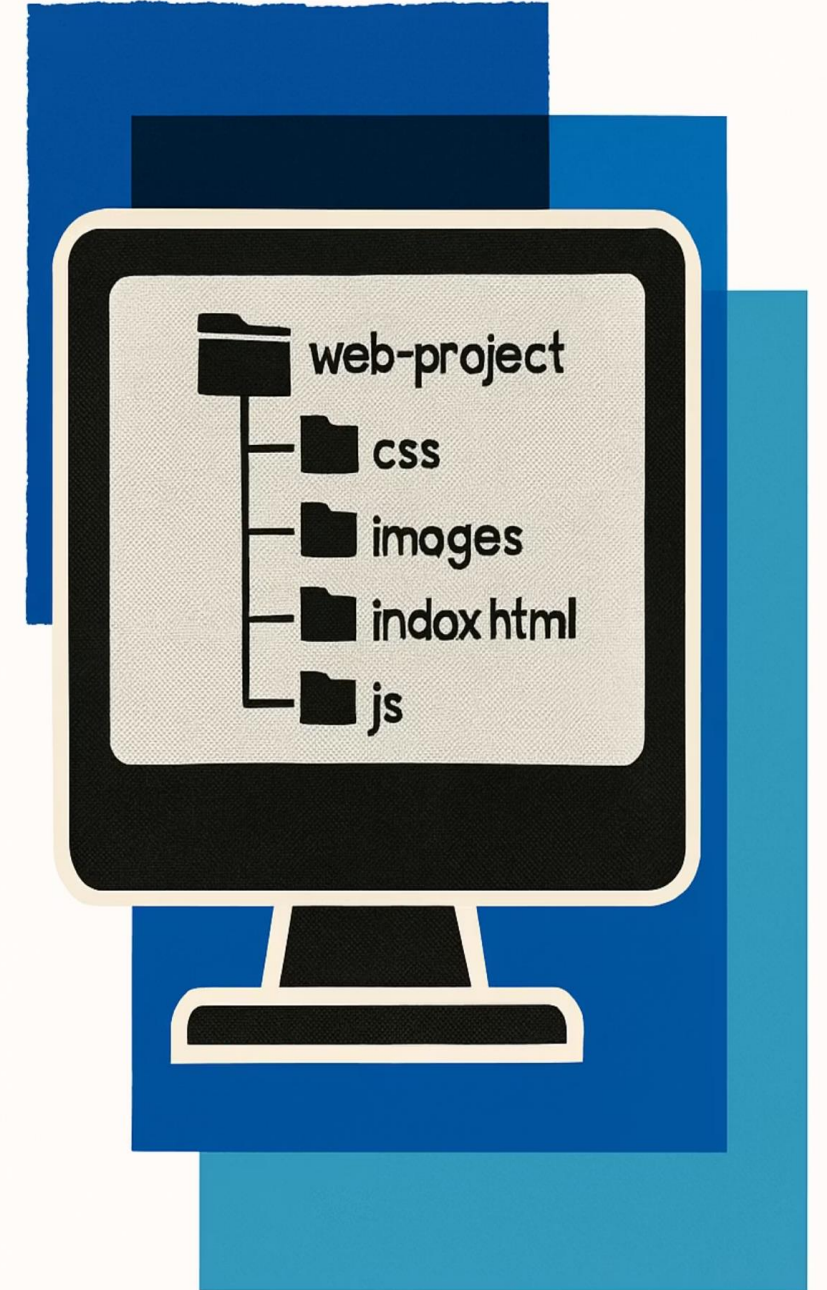
- Componente `produto` (filho)
- Componente `lista-produtos` (pai)
- Comunicação via `@Input` e `@Output`

Estrutura de Pastas

- `core/`
- `shared/`
- `features/produtos/`

Estado Atual

- Lista de produtos funcionando com `@for`
- Dados armazenados em **variável comum** (array simples)
- Sem reatividade automática



O Problema Atual

⚠ O estado atual do projeto **NÃO é reativo**. O Angular não sabe automaticamente quando o array muda. Ele precisa de uma "notificação" para atualizar a tela.

O que acontece hoje

Temos um array simples declarado no componente:

```
produtos = [  
  { nome: 'Notebook', preco: 3800 },  
  { nome: 'Mouse', preco: 179 }  
];
```

Se fizermos `produtos.push(...)`, o Angular **pode não perceber** a mudança a tempo — ou pode ser ineficiente.

Por que isso é um problema?

- Alterações no array **não são rastreadas** automaticamente
- Em versões antigas, isso exigia **RxJS** (complexo para iniciantes)
- A UI pode ficar desatualizada em cenários mais complexos
- Código menos previsível e mais difícil de manter

④ Em aplicações simples pode parecer funcionar, mas em cenários reais (listas grandes, múltiplos componentes ou performance otimizada), o Angular pode não atualizar corretamente.

O que são Signals?

Signal é um valor especial que o Angular consegue **observar automaticamente**. Quando o valor muda, qualquer parte do template que o utiliza é atualizada sem que você precise fazer nada.



Reatividade automática

O Angular "ouve" o signal.
Quando o valor muda, a tela é atualizada instantaneamente.



Sem RxJS

Signals foram criados para ser mais simples que Observables. Iniciantes conseguem usar sem conhecer RxJS.



Atualizações precisas

O Angular atualiza **apenas** o que mudou, tornando a aplicação mais eficiente.

Variável Comum vs. Signal

Veja a diferença fundamental entre os dois modelos:

ANTES — Variável comum

```
// Declaração
produtos = [
  { nome: 'Notebook', preco: 3800 }
];

// Modificação (sem rastreamento)
this.produtos.push(
  { nome: 'Mouse', preco: 179 }
);

// Template
@for (item of produtos; ...)
```

O Angular **não sabe** que o array foi alterado de forma garantida.

DEPOIS — Signal

```
// Declaração
produtos = signal([
  { nome: 'Notebook', preco: 3800 }
]);

// Modificação (rastreada!)
this.produtos.update(lista => [
  ...lista,
  { nome: 'Mouse', preco: 179 }
]);

// Template
@for (item of produtos(); ...)
```

O Angular **detecta automaticamente** toda alteração.

A Regra do ()

Por que existe o ()?

Um `signal` não é um valor direto. Ele é uma **função que retorna o valor**. Para obter o conteúdo, você precisa chamá-lo:

```
// ERRADO
@for (item of produtos; track item.nome)
```

```
// CORRETO
@for (item of produtos(); track item.nome)
```

Sem os parênteses, o Angular lança erro e a tela quebra.

❌ REGRA FUNDAMENTAL (Leia com atenção!)

Signals são acessados como função: você DEVE usar parênteses `()` para ler o valor.

Parte Prática

Mão na Massa — 8 Passos

Agora vamos aplicar tudo o que aprendemos diretamente no nosso projeto de e-commerce.

Siga **cada passo com calma** — não pule nenhuma etapa!



1 - Parar o Servidor

Por que parar o servidor?

Sempre que vamos fazer alterações em arquivos de configuração ou mudanças estruturais no projeto, é uma boa prática parar o servidor para evitar conflitos e garantir que as mudanças sejam carregadas corretamente ao reiniciar.

Como parar

No terminal onde o servidor está rodando, pressione:

CTRL + C

Como saber se parou?

O terminal voltará a exibir o **prompt normal** (o cursor piscando aguardando um novo comando). Se ainda mostrar logs do Angular, o servidor ainda está rodando. Repita o CTRL + C.

2 - Importar o `signal`

Arquivo a editar: `features/produtos/lista-produtos/lista-produtos.ts`

ANTES

```
import { Component } from '@angular/core';
```

Apenas o `Component` estava importado — suficiente até agora, mas não para usar Signals.

DEPOIS

```
import { Component, signal }  
  from '@angular/core';
```

Adicionamos `signal` à lista de importações. Ele é uma **função nativa do Angular** — não precisa instalar nada extra!

- ❗ Dica: Sempre verifique se o nome está exatamente correto: `signal` com letra minúscula. O VSCode geralmente autocompleta para você.

3 - Transformar o Array em Signal

Ainda no arquivo `lista-produtos.ts`, vamos converter a declaração dos produtos:

ANTES — Array simples

```
produtos = [  
  { nome: 'Notebook', preco: 3800 },  
  { nome: 'Mouse',    preco: 179 }  
];
```

Um array normal — estático, sem rastreamento pelo Angular.

DEPOIS — Signal reativo

```
produtos = signal([  
  { nome: 'Notebook', preco: 3800 },  
  { nome: 'Mouse',    preco: 179 }  
]);
```

Agora `produtos` é um **signal**. O Angular passa a rastrear qualquer mudança nesse valor automaticamente.

- ❏ Perceba: o conteúdo do array não mudou — apenas **embrulhamos** o array dentro de `signal()`.
Os dados são os mesmos!

4 - Ajustar o Template (@for)

Arquivo: lista-produtos.html

ANTES

```
@for (item of produtos;  
      track item.nome) {  
  ...  
}
```

Lia `produtos` diretamente como array.

DEPOIS

```
@for (item of produtos());  
      track item.nome) {  
  ...  
}
```



Agora chamamos `produtos()` com parênteses, porque `signal` é uma função. Isso retorna o array para o `@for` iterar.

⊗ **ALERTA CRÍTICO:** Esquecer os `()` após `produtos` no template é o erro mais comum desta aula. O Angular não consegue iterar sobre um signal sem os parênteses e vai exibir **erro no console do navegador**.

5 - Criar o Método `adicionarProduto()`

Adicione o método a seguir dentro da classe do componente, no arquivo `lista-produtos.ts`:

```
adicionarProduto() {  
  this.produtos.update(listaAtual => [  
    ...listaAtual,  
    { nome: 'Teclado', preco: 250 }  
  ]);  
}
```

`update()`

Método do signal para atualizar o valor. Recebe uma função com o valor atual e retorna o novo valor.

`...listaAtual`

O operador spread copia todos os itens existentes, garantindo **imutabilidade** (não modificamos o array original).

Reatividade

Após o `update()`, o Angular detecta a mudança automaticamente e re-renderiza o template.

6 - Adicionar o Botão no Template

No arquivo `lista-produtos.html`, adicione o botão abaixo. Pode ser antes ou depois do `@for`:

```
<p><button (click)="adicionarProduto()">Adicionar Produto</button></p>
```

O que é `(click)`?

É a sintaxe do Angular para **event binding**, ou seja, ligar um evento do DOM (o clique do usuário) a um método do componente. Sempre usa parênteses `()` ao redor do nome do evento.

O que vai acontecer?

Quando o usuário clicar no botão, o método `adicionarProduto()` será chamado. Ele usa `update()` para adicionar "Teclado" ao signal e o Angular atualiza a lista na tela **automaticamente**.

✅ O clique dispara uma mudança no signal — e isso automaticamente atualiza a interface.

7 - Reiniciar o Servidor

Execute no terminal:

```
ng serve
```

Aguarde até aparecer a mensagem confirmando que o servidor está rodando (geralmente em `http://localhost:4200`).

Por que reiniciar?

- Garante que **todas as alterações** foram carregadas corretamente
- Descarta qualquer cache de compilação que possa causar comportamento inesperado
- Sempre uma boa prática após mudanças estruturais no TypeScript



Se o servidor compilar sem erros, você está no caminho certo! Erros de compilação aparecerão diretamente no terminal. Leia-os com atenção.



8 - Testar no Navegador

1 Abra `localhost:4200`

A lista de produtos deve aparecer normalmente, com Notebook e Mouse, exatamente como antes.

2 Clique no botão "Adicionar Produto"

Um novo item — o **Teclado (R\$ 250)** — deve aparecer na lista instantaneamente, sem recarregar a página.

3 Clique novamente

A cada clique, um novo Teclado é adicionado. Isso confirma que o signal está funcionando corretamente!

O Que Acabou de Acontecer?

Vamos consolidar o que foi feito e entender o impacto real dessa mudança no projeto.



Estado reativo

O array de produtos agora é um signal — o Angular rastreia cada mudança automaticamente.



UI automática

Não precisamos chamar nenhuma função de atualização manual — o template se atualiza sozinho.



Sem RxJS

Conseguimos reatividade completa sem usar Observables, Subjects ou qualquer operador RxJS.



Nova mentalidade

Passamos de "eu controlo a atualização" para "eu declaro o estado e o Angular cuida do resto".

Erros Comuns — Fique Atento!

⊗ Estes são os erros que **todo iniciante comete** nesta etapa. Salve este slide para consultar quando algo não funcionar!

1. Esquecer o `()` no template

`@for (item of produtos; ...)` — sem os parênteses, o Angular não sabe que deve chamar a função do signal. Use sempre `produtos()`.

2. Usar `push()` ao invés de `update()`

`this.produtos().push(...)` modifica o array interno sem notificar o signal. O Angular **não detecta** essa mudança. Use sempre `this.produtos.update(...)`.

3. Tentar misturar RxJS

Signals e Observables são mundos diferentes.

Nesta aula não usamos RxJS. Não tente combinar os dois sem entender a integração correta.

4. Não reiniciar o servidor

Após mudanças no `.ts`, especialmente em imports, sempre reinicie com `ng serve` para garantir compilação limpa.

Comparação Didática

Veja lado a lado tudo o que mudou ao adotar Signals no lugar de variáveis comuns:

Aspecto	Variável Comum	Signal
Declaração	<code>produtos = [...]</code>	<code>produtos = signal([...])</code>
Leitura no template	<code>produtos</code>	<code>produtos()</code>
Atualização	<code>produtos.push(...)</code>	<code>produtos.update(...)</code>
Angular detecta mudança?	Nem sempre	Sempre
Precisa de RxJS?	Em casos complexos, sim	Não
Imutabilidade	Manual (atenção do dev)	Guiada pelo <code>update()</code>

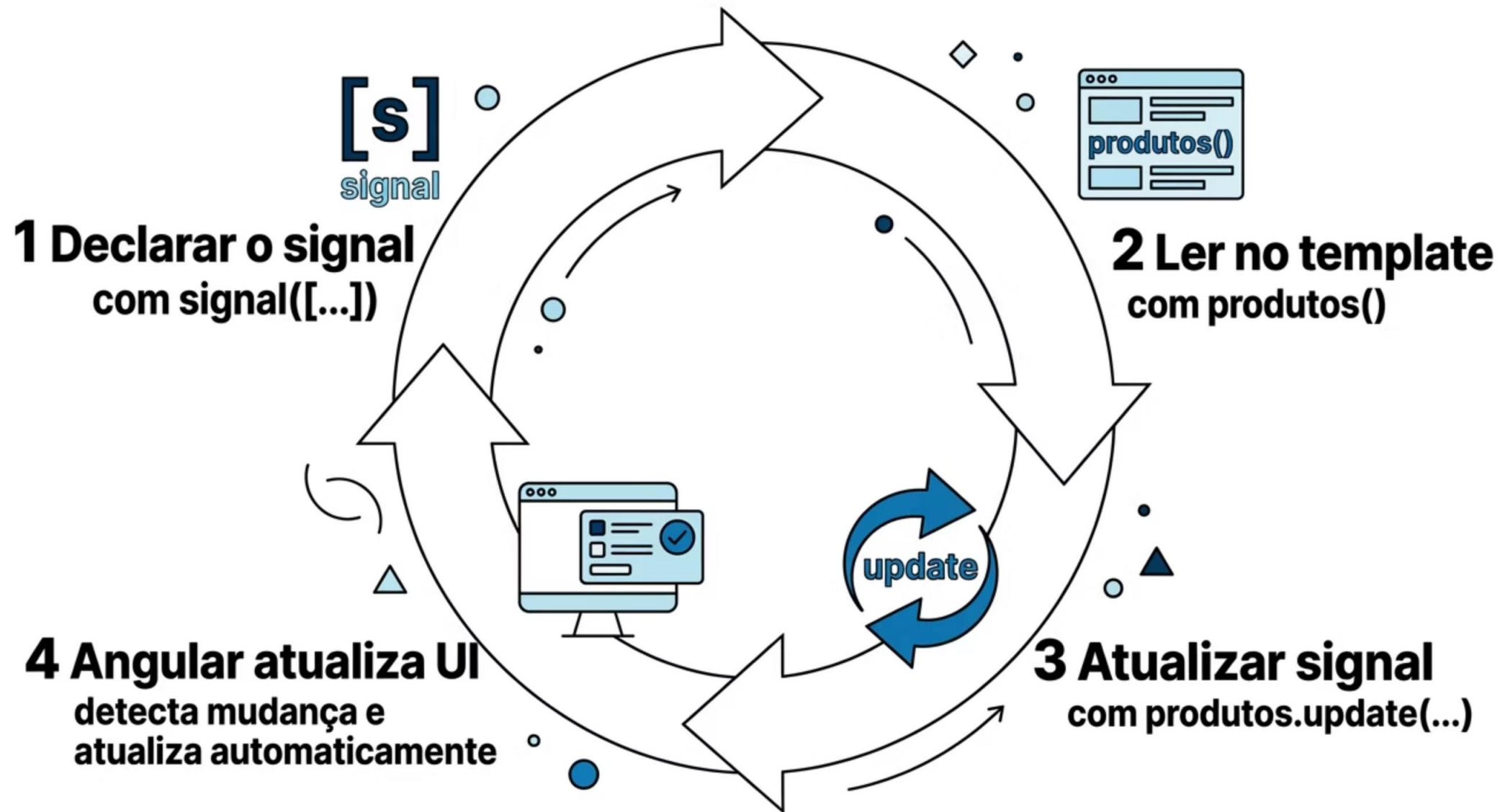
Conexão com as Aulas Anteriores

Cada aula do curso constrói sobre a anterior. Veja como chegamos até aqui:

- 1 — Aula 06
Comunicação entre components. Aprendemos `@Input` e `@Output` para passar dados entre pai e filho.
- 2 — Aula 07
Organização do projeto. Estruturamos o projeto em pastas `core/`, `shared/` e `features/`.
- 3 — Aula 08
Gerenciamento de estado. Agora o estado é reativo com `signal()` — o Angular gerencia as atualizações.
- 4 — Próximas aulas
Mais funcionalidades. Continuaremos evoluindo o e-commerce com novos conceitos modernos.



O Ciclo do Signal



Esse ciclo é a essência dos Signals: declarar, ler, atualizar — e deixar o Angular fazer o resto.

Checklist Final da Aula

Antes de fechar o projeto, confirme que você completou cada etapa:

- ✓ Parei o servidor com CTRL + C
- ✓ Adicionei `signal` ao import do `@angular/core`
- ✓ Transformei o array em `signal([...])`
- ✓ Atualizei o template: `produtos()` com parênteses no `@for`
- ✓ Criei o método `adicionarProduto()` usando `update()`
- ✓ Adicionei o botão com `(click)="adicionarProduto()"` no HTML
- ✓ Reiniciei o servidor com `ng serve`
- ✓ Testei no navegador e vi o novo produto aparecer ao clicar

👉 Se todos os itens estão marcados, parabéns! Você acabou de implementar estado reativo com Signals no Angular moderno. 🎉

Você Concluiu!

Hoje demos um salto importante na qualidade do nosso projeto de e-commerce. Saímos de um estado estático e passamos a ter uma aplicação verdadeiramente reativa, do jeito Angular moderno.

O que você aprendeu

- O conceito de Signals
- Como declarar e usar `signal()`
- A regra do `()` no template
- Como atualizar com `update()`

O que o projeto ganhou

- Estado reativo e eficiente
- UI que se atualiza automaticamente
- Código mais limpo e previsível
- Sem necessidade de RxJS

Próxima aula

Continuaremos evoluindo o projeto, aprofundando em Signals com `computed()` e `effect()` — ainda mais poder, ainda sem RxJS.



Dúvidas

?

